SportMaps

GUÍA PARA DESARROLLO · USO INTERNO

Probar con Docker: el gemelo local de la base

Supabase completo corriendo en tu máquina, con el **esquema real** de la base viva y **datos de prueba**. Ahí corren las pruebas de RLS, de concurrencia y de Playwright sin tocar producción.

Para: Julián | Windows + Docker Desktop | Primera vez ~15 min · después ~40 s

Versión del 14 de octubre de 2026 · Fuente técnica completa en el repo: docs/qa-gemelo-local.md

0 Por qué existe

Hay **una sola Supabase** (`luebjarufsiadojhvxgi`) para dev, staging y producción. Cualquier prueba que escriba en ella escribe en datos reales de escuelas y familias. Además, unos 336 objetos de la base viva no están en `supabase/migrations/`, así que levantar Supabase con las migraciones del repo da **otra** base distinta.

El gemelo resuelve las dos cosas: copia **solo el esquema** de lo que la viva tiene hoy (tablas, funciones, policies, triggers y permisos) y le carga una semilla sintética. **Nunca** trae personas, pagos, escuelas ni documentos reales.

> **REGLA DE ORO**
> Las pruebas que escriben o que validan permisos corren **contra el gemelo**, nunca contra la base viva. Las guardas del repo abortan la corrida si detectan la URL de producción (sección 7).

1 Instalar (una sola vez)

1 Docker Desktop

Instalar Docker Desktop con el motor **WSL 2** y dejarlo abierto. Comprobar: `docker info` debe responder sin error. Necesita ~4 GB libres para las imágenes.

2 Node 20+ y la CLI de Supabase

```
node -v                     # 20 o más
scoop install supabase      # o: npm i -g supabase
supabase --version
```

3 Repo y dependencias

```
git clone https://github.com/brylop/sportmaps-demo.git
cd sportmaps-demo
git checkout develop
npm install
cd bff; npm install; cd ..
cd frontend; npm install; npx playwright install chromium; cd ..
```

2 Conseguir el esquema

Va en supabase/qa-twin/dump/ · git lo ignora a propósito

El gemelo se arma con un volcado del esquema de la viva. Hay dos caminos:

A · RECOMENDADO

Pedirle a Brayan el **.zip del volcado** y descomprimirlo en `supabase/qa-twin/dump/`. Son 4 archivos `.sql` más `manifest.json`. Solo esquema y catálogos, sin datos personales.

B · SOLO SI TIENES ACCESO

Con acceso al proyecto de Supabase: `supabase login`, luego `supabase link --project-ref luebjarufsiadojhvxgi` y `npm run qa:dump-schema`. Es de solo lectura.

OJO CON LA BASE VIVA

El volcado lee todo el catálogo de la base viva, y eso consume el presupuesto de disco que el 14 de octubre tumbó la plataforma ~14 horas. Generarlo **solo cuando cambie el esquema**, nunca en bucle, y nunca en paralelo con migraciones.

3 Levantar el gemelo

1 Arrancar

```
npm run qa:twin:up
```

La primera vez baja ~2 GB de imágenes (~4 min). Si la base está vacía, carga el volcado y la semilla. Si Docker Desktop estaba apagado, el script intenta abrirlo.

2 Ver URL y llaves

```
npm run qa:twin:status
```

Imprime la URL, las llaves `anon` / `service_role` locales (son las de demo de la CLI, iguales en toda máquina, no son secretas) y la cadena de conexión.

3 Verificar que es igual a la viva

```
npm run qa:twin:verify
```

Compara relaciones, funciones, policies, triggers y **permisos** de `anon` / `authenticated`. Sale con error si difieren. Una prueba negativa de RLS solo vale si el gemelo tiene los mismos permisos que la viva.

Servicio	Dirección local
API (`SUPABASE_URL`)	`http://127.0.0.1:54321`
Postgres	`postgresql://postgres:postgres@127.0.0.1:54322/postgres`
Studio	`http://127.0.0.1:54323`
Correos (Mailpit)	`http://127.0.0.1:54324`

4 Usuarios y datos de prueba

Contraseña de todos: QaGemelo2026! · Correo: <alias>@qa.sportmaps.test

Alias	Rol	Alias	Rol
`owner.a`	Owner de **QA Academia Andes** (pro, tienda + contabilidad)	`owner.b`	Owner de **QA Club Llanos** (starter, sin addons)
`admin.a`	school_admin de A	`padre.b`	Padre **ajeno** a A (hijo en B)
`coach.a`	Coach de A	`vendedor.ok`	Vendedor externo verificado
`padre.a`	Padre miembro de A	`vendedor.pend`	Vendedor sin verificar
`atleta.a`	Atleta adulto de A	`superadmin`	Admin de plataforma

Hay además 7 productos (con variantes, agotados, en borrador), cobros `paid` / `partial` / `pending`, proveedor con factura, gastos y un empleado de nómina. Los IDs son fijos; desde SQL: `select * from qa_twin.actores`.

5 Correr las pruebas

1 Casos SQL de RLS y permisos

```
npm run qa:sql -- supabase/tests/tienda_v2          # una carpeta
npm run qa:sql -- supabase/tests/tienda_v2/R03_anon_no_lee_bank_data.sql
```

Un archivo es un caso, todo dentro de `BEGIN … ROLLBACK`. Pasa si termina sin error. Hay carpetas para tienda, contabilidad, hermanos y Dreamers en `supabase/tests/`.

2 Vitest de integración del BFF (concurrencia)

```
cd bff
$env:SUPABASE_URL="http://127.0.0.1:54321"
$env:WOMPI_PUBLIC_KEY="pub_test_x"
$env:PUBLIC_API_URL="http://127.0.0.1:3000"
npx vitest run -c vitest.integration.config.ts
```

Sin esas variables la guarda aborta, porque `bff/.env` apunta a la viva. Es a propósito.

3 Playwright contra el gemelo

```
cd frontend
npx playwright test -c playwright.gemelo.config.ts
```

Levanta **su propio** Vite en el puerto 3101 con las variables del gemelo. No reutiliza un `npm run dev` abierto en 3001, porque ese apunta a la viva y la UI escribiría en producción.

LAS UNITARIAS NO CAMBIAN

`npx vitest run` dentro de `bff/` (archivos `src/**/*.test.ts`) usa mocks y no necesita el gemelo ni Docker.

4 Escribir un caso SQL nuevo

```
begin;
select set_config('request.jwt.claims', json_build_object(
  'sub', (select user_id from qa_twin.actores where alias = 'padre.a'),
  'role','authenticated')::text, true);
set local role authenticated;
do $$ begin
  begin
    insert into public.orders(user_id,total_amount,status) values (auth.uid(),1000,'paid');
    raise exception 'FALLO: el comprador inserta una orden pagada';
  exception when insufficient_privilege then raise notice 'OK: 42501';
  end;
end $$;
rollback;
```

Leer el actor **antes** de bajar de rol: `authenticated` no ve el esquema `qa_twin`.

6 Probar una migración antes de aplicarla

1 Aplicarla en el gemelo

```
Get-Content supabase\migrations\2026XXXX_mi_cambio.sql |
  docker exec -i supabase_db_sportmaps-qa-twin psql -U postgres -v ON_ERROR_STOP=1
```

2 Correr los casos y la seguridad

Correr los casos SQL del módulo y, si tocaste RLS o permisos, confirmar que no sale nada crítico en `select * from public.invariantes_seguridad()` dentro del gemelo.

3 Volver al estado de la viva

```
npm run qa:twin:reset      # base en blanco + volcado + semilla (~1 min)
```

LA MIGRACIÓN A LA VIVA VA APARTE

Probar en el gemelo no la aplica en producción. En la viva la aplica **una sola persona a la vez** y por una vía que deje rastro; coordinarlo con Brayan. Las reglas completas están en el `CLAUDE.md` del repo.

7 Guardas anti-producción

Estas guardas abortan la corrida si cualquier URL o cadena de conexión contiene `luebjarufsiadojhvxgi`, o si la llave de Wompi empieza por `pub_prod_`:

Dónde	Archivo
Playwright	`frontend/e2e/helpers/guard-no-prod.ts`
Vitest de integración	`bff/test/guard-no-prod.ts`
Scripts del gemelo	`scripts/qa-twin/lib.mjs` (solo localhost)

Si una corrida aborta por la guarda, **no la desactives**: falta exportar las variables del gemelo.

8 Problemas comunes

Síntoma	Qué hacer
`500 Internal Server Error … images` al primer arranque	Docker Desktop recién encendido. Repetir `npm run qa:twin:up`
`qa:twin:verify` difiere justo después de un reset	La viva cambió desde el volcado, o hay un objeto que el volcado no cubre. Revisar `supabase/qa-twin/dump/load.log` y pedir un volcado nuevo
Un producto no se puede crear directo en `active`	Comportamiento real de la base: insertar en `draft` y luego cambiarlo a `active`
Un vendedor externo no puede editar productos	Comportamiento real (policies `trial_block_*` con escuela nula). No es un problema del gemelo
Correos de prueba	No salen a internet: se ven en Mailpit, `http://127.0.0.1:54324`
No hay crons ni edge functions	El gemelo no los replica. `pg_net` existe, pero sus llamadas no llegan a ningún lado

✓ Comandos del día a día

Comando	Para qué
`npm run qa:twin:up`	Arrancar (~40 s si ya estaba cargado)
`npm run qa:twin:status`	URL, llaves y conexión
`npm run qa:twin:verify`	¿Sigue igual a la viva?
`npm run qa:sql -- <archivo|carpeta>`	Casos SQL
`npm run qa:twin:seed`	Volver a cargar solo la semilla
`npm run qa:twin:reset`	Dejarlo como recién cargado (~1 min)
`npm run qa:twin:down`	Apagar. Los datos quedan en el volumen